

When Does Context Pay Off?

TF-IDF, Word2Vec and BERT for sentiment classification under label, compute and domain constraints

Mohammed Jaffer Siddique Abdul Jabbar · Student ID 1908893 University of Trier · 2026

1 — HYPOTHESIS

*The ranking $TF\text{-}IDF < Word2Vec < BERT$ is established on large, clean, in-domain benchmarks. **Under what conditions does that ranking actually hold — and where does it break?***

Text representation has moved from **simple word counts** (TF-IDF) to **dense word vectors** (Mikolov et al., 2013; Pennington et al., 2014) to **context-aware embeddings** (Devlin et al., 2019), and each step has been sold as a clear win. But those wins are usually measured under the same generous conditions: plenty of labelled data, no compute budget, and test data that looks just like the training data. Few real projects get all three at once.

So here we keep the classifier identical throughout and change only the representation, testing each one against the constraints a real project would actually face.

H1 · LABEL EFFICIENCY

Contextual embeddings should only win once past a certain label budget. Below roughly 500 labelled reviews, we expect plain TF-IDF to hold its own or do better, since it keeps the exact words (*terrible*, *waste*) that pooling tends to blur together.

H2 · COMPUTE

The accuracy bought per unit of compute should fall off sharply — TF-IDF sitting on the accuracy/latency frontier, and BERT not.

H3 · ROBUSTNESS

BERT's real advantage should show up specifically on negation, contrast and out-of-domain text — meaning the standard benchmark number actually *understates* where context genuinely helps.

2 — METHODOLOGY

One variable: the representation

■ TF-IDF ■ Word2Vec / GloVe ■ DistilBERT

Condition	Type	Dim
TF-IDF	sparse, 1–2 gram	50k
Word2Vec (in-domain)	static, skip-gram	300
GloVe (pretrained)	static, pretrained	300
DistilBERT [CLS]	contextual, frozen	768
DistilBERT mean-pool	contextual, frozen	768

Every condition is fed into the same **logistic-regression classifier**, with the same hyperparameter search on the same dev split — so any accuracy difference can only come from the representation itself.

BERT here is frozen, not fine-tuned. Fine-tuning would blur together a better representation with simply more trainable parameters, so freezing isolates the representation — at the cost of putting BERT's accuracy at a floor, not a ceiling (85.4–86.4% here vs. roughly 93% fine-tuned). If anything this favours BERT for H2: fine-tuning only pushes its cost further right, never left.

Data & protocol

Corpus	Role	n
IMDB (Maas et al., 2011)	train pool	20,000
IMDB held-out	dev — selects C only	5,000
IMDB test	evaluation, touched once	25,000
Rotten Tomatoes	OOD — length shift	1,066
Twitter US Airline	OOD — domain shift	~3,000

Long IMDB reviews are deliberately the toughest setting for BERT's 256-token window. In total: 8 label budgets × 5 representations × 5 seeds = **200 model fits**, all reusing features encoded once and cached — the only reason this is feasible on a CPU.

3 — RESULTS

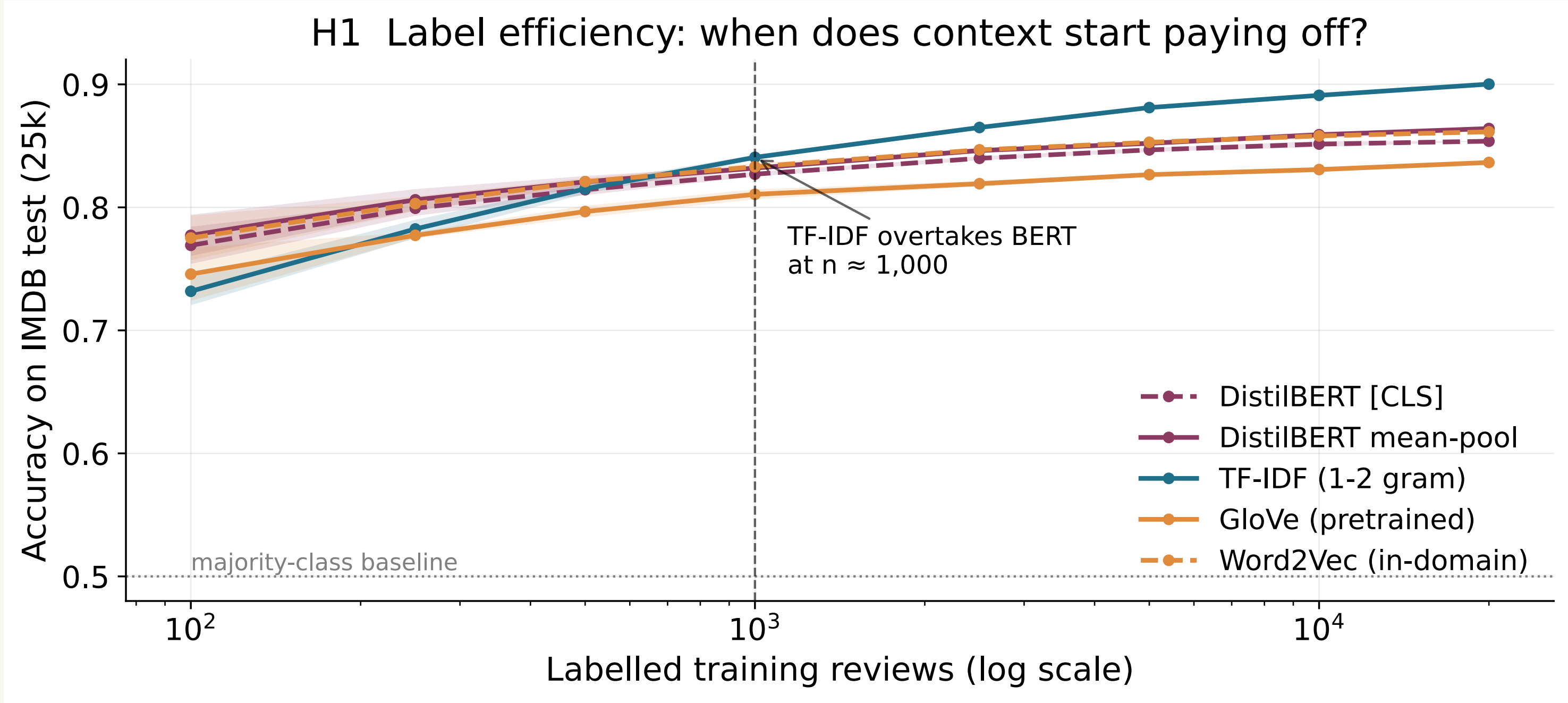


Fig 1 · H1. Accuracy on the full IMDB test set vs. labelled training reviews. Bands are 95% CIs over 5 seeds.

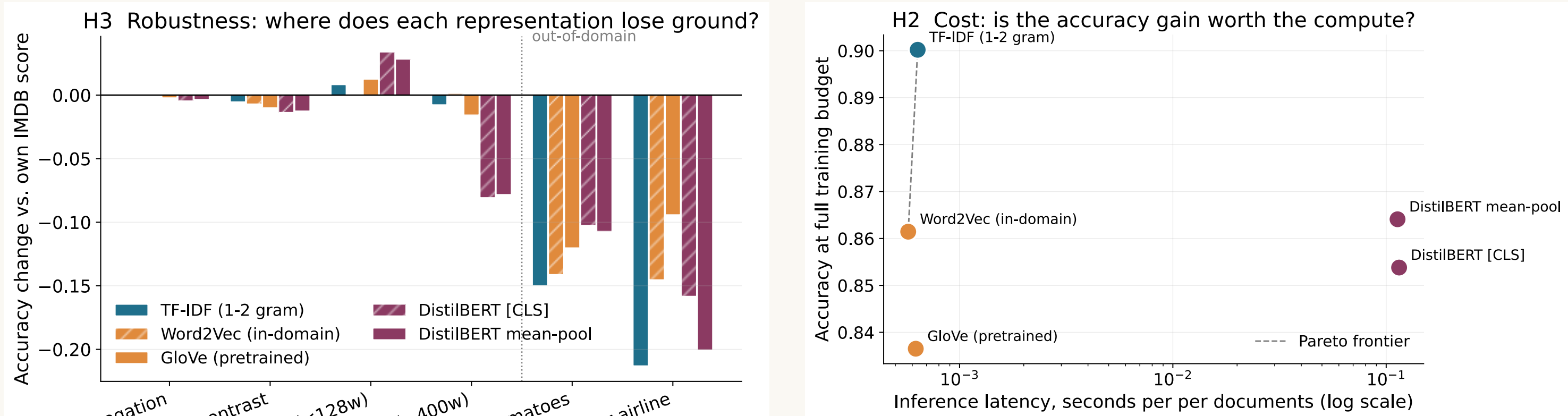


Fig 2 · H2. Accuracy against end-to-end inference latency (log scale), at the full training budget.

Fig 3 · H3. Accuracy change relative to each representation's own IMDB score, by test slice.

Findings

H1 flips on us: dense methods lead at 100 labels, but TF-IDF overtakes everything by ~1,000 examples and climbs to **90.0%** at 20,000 — past frozen BERT's 85–86% ceiling. **H2 holds up cleanly:** BERT costs ~180× more per document for accuracy that's no better. **H3 is mixed:** negation barely moves any method, but long reviews hurt BERT (–7.8 to –8.1pp vs. TF-IDF's –0.8pp), while Twitter's domain shift hits TF-IDF hardest (–21.3pp).

Limitations

Because BERT is frozen, these numbers are a floor, not a ceiling — TF-IDF beats a frozen feature here, not context itself. The 256-token limit disadvantages BERT on long reviews, and pooling discards word order, which may explain negation's null result. Training is single-domain; OOD results are zero-shot.